Tweet Sentiment Label Quality: A Text Classification Analysis

David Ewing · Student ID: 82171165

1 Problem Statement

This project examines how well an automated text classifier can reproduce the sentiment labels applied to a large sample of tweets. The dataset is the Cardiff NLP tweet_eval sentiment subset — 45,615 tweets labelled as negative, neutral, or positive by human annotators. Labelling was conducted as part of the SemEval 2017 Task 4A shared task on sentiment analysis in Twitter (Rosenthal et al., 2017), and the labels are widely used as an NLP benchmark.

My central question is: how reliably can a text classifier reproduce these labels, and what does classifier performance reveal about the quality and consistency of the annotations? This matters because weak classification performance may reflect not just the difficulty of the task but genuine inconsistency in how the labels were assigned. A classifier that performs poorly on neutral tweets may be revealing that the boundary between neutral and mildly positive or mildly negative is genuinely ambiguous — that different annotators drew that line differently. Put differently, low F1 may be evidence of noisy labels as much as evidence of an inadequate model. Both can be true at once.

Four sub-questions guide the analysis:

1. What kinds of problem are present in the labelling of tweets?
2. What types of tweet are systematically hard to classify?
3. Which features are most informative for distinguishing sentiment classes?
4. Does removing the neutral class substantially improve classifier performance, and what does this imply about the consistency of the neutral label?

Annotated datasets are expensive to produce. My sense is that once a benchmark is established, it tends to be used repeatedly — models are compared to each other using the same labels, and the labels themselves are rarely questioned. If I follow that reasoning, gold-standard labels that contain systematic noise mean every model comparison on that benchmark inherits that noise. A model that achieves a higher F1 than a competitor may simply be better at reproducing the particular pattern of annotator disagreement baked into the dataset, not better at genuine sentiment classification. That is why I think being explicit about label quality matters — it is a methodological obligation, not just an academic nicety.

I chose macro average F1 as the primary evaluation metric. The validation split (2,000 tweets) serves as the test set throughout. Because the training data is class-imbalanced — neutral tweets account for roughly 45% of the full training set — random undersampling is applied to reduce all classes to the size of the smallest (7,093 negative tweets), producing a balanced training set of 21,279 examples. I preferred undersampling over oversampling because I wanted to avoid synthesising or duplicating examples that might reinforce noisy labels. Macro F1 averages equally across all three classes, rather than being dominated by the majority class. I chose it because the research question treats all three classes as equally important: a metric that concentrates weight on neutral tweets would obscure precisely the per-class performance differences this analysis is designed to detect. Per-class precision and recall are also reported. I avoided accuracy as the primary metric because it rewards over-predicting the majority class.

2 Dataset

The dataset is the sentiment subset of the Cardiff NLP tweet_eval benchmark (Barbieri et al., 2020). It was collected from Twitter and annotated through the SemEval 2017 Task 4A competition, where teams competed to classify tweet sentiment. Labels — negative, neutral, and positive — were assigned by crowdsourced annotators, with final labels reflecting majority vote across multiple annotations.

Crowdsourced annotation means individual workers on a platform like Amazon Mechanical Turk are each shown a tweet and asked to assign a label (Rosenthal et al., 2017). Each tweet typically receives three or more annotations, and the final label is the majority vote. This is efficient, but it carries a hidden cost. Annotators may disagree. On close calls — a mildly sarcastic tweet, or an expression of resignation that could be read as neutral or mildly negative — the majority vote will still produce a label, but that label may not reflect genuine consensus. I think this is important context for interpreting classifier performance. When a model gets a tweet "wrong", I think it is worth asking whether the gold label itself was uncertain. Sometimes the answer is yes.

After applying random undersampling (random_state=82171165) to balance the training classes, the training set is reduced to 21,279 tweets — 7,093 per class. The validation split (2,000 tweets) is used as the evaluation set throughout this report. The test split (12,284 tweets) is held out and not used. Exact per-class counts at each stage of processing are shown in notebook section 2.5.

The class distribution in the validation set is unbalanced: 312 negative, 869 neutral, and 819 positive tweets. This matters. The negative class has fewer than half the examples of positive, and less than 40% of neutral. When I first noticed this, I realised it would make the negative per-class results harder to interpret. Poor performance on negative tweets might reflect label noise, or a weak feature set, or simply the fact that there are fewer test examples to evaluate on. Probably all three.

A further consequence of the unbalanced test set is that confidence intervals around the per-class estimates differ substantially. An F1 estimate based on 312 examples carries wider error bars than one based on 869. When I compare negative F1 to neutral F1 and find a gap, I have to be honest about whether that gap is meaningful or within the noise

introduced by the smaller sample. This is not a reason to ignore the difference, but it is a reason to interpret it carefully.

Tweets are short, informal, and noisy: hashtags, @-mentions, URLs, slang, emoji, and abbreviations are common. They are also highly context-dependent. The same words can carry different sentiment depending on the event being discussed, who is being addressed, and whether the author is being ironic. A tweet about a sports loss might use the word "great" sarcastically — I noticed examples like this in my manual preview. A tweet directed at a celebrity might use "love" in a way that is enthusiastic. My sense is that these properties make sentiment classification in Twitter data harder than in product reviews or news text, where context is more stable and language use more predictable, though I did not run a direct comparison.

I previewed a random sample of 10 training tweets during the exploratory phase. Even with human reading, many were genuinely ambiguous. One tweet about a concert announced a future event in neutral descriptive language but was labelled positive. I see this as defensible — the author clearly approved of the event — but a strict reading of the text gives no obvious signal. Another tweet expressed mild frustration using indirect language — not swearing, not shouting, just a brief complaint — and was labelled neutral. I would have called it negative. These kinds of borderline cases are unavoidable in Twitter data, and I expect the classifier to struggle on them for the same reasons a careful human reader might pause.

3 Model

Two classifiers are evaluated across six feature configurations and two task variants, producing four runs in total. The classifiers are logistic regression and a shallow decision tree. Both use the same scikit-learn pipeline, fitted to the balanced training set and evaluated on the validation split. The pipeline is shown in Figure 1.

The pipeline has four fixed steps. First, a TextCleaner component normalises whitespace. Second, a SpacyPreprocessor tokenises each tweet using the en_core_web_sm spaCy model and caches results in an SQLite feature store. Third, a FeatureUnion assembles the feature matrix from the active feature blocks. Fourth, the feature matrix is passed to the classifier. This architecture makes it easy to swap feature configurations while holding preprocessing constant.

Logistic regression is configured with max_iter=5000, random_state=42. The decision tree uses max_depth=3, random_state=42. The shallow depth restricts the tree to at most seven leaf nodes, which forces it to identify only the most discriminating features and produces a model small enough to visualise and interpret directly.

The choice to constrain the decision tree rather than tune it for performance was deliberate. A deep tree would almost certainly outperform a tree with three levels. But the interpretive goal of this analysis is to understand what features the classifier uses, not to maximise accuracy. A seven-node tree can be printed, examined, and explained. The max_depth=3 setting is a diagnostic tool as much as a classifier. The cost of this choice in performance terms is real, and I discuss it in Section 5.

3.1 Feature configurations

Six feature configurations are tested. They are designed to progress from simple token-count representations through to richer, more linguistically informed features, so that each step in complexity can be assessed against the baseline.

Unigrams. A TokensVectorizer extracts count features for the top 200 most frequent unigram tokens, after removing stop words and punctuation, with lowercasing. This is the simplest possible bag-of-words representation. It captures nothing about word order, negation, or context. I expected this to perform poorly, and it does — but it is still a meaningful baseline because it tells us what a purely lexical, context-free representation can do. Any result worse than this is a failure in a meaningful sense.

Bigrams. A TokensVectorizer extracts the top 200 most frequent bigrams. Bigrams capture some local context — "not good" is different from "good" — but they remain shallow and sensitive to data sparsity. Twitter language is varied enough that any specific bigram may appear rarely. A bigram like "really love" might appear hundreds of times in the training set, but "really enjoy" or "absolutely love" might appear only once each. The top-200 constraint means only the most frequent bigrams are captured, and Twitter users are inventive enough that the most frequent bigrams may not be the most sentiment-informative.

Unigrams with POS tags (uni+pos). A FeatureUnion combines unigram token counts (top 100 features, scaled) with POS tag sequence features from a POSVectorizer. The intuition is that grammatical structure may carry sentiment signal that vocabulary alone misses. Intensifiers, negations, and adjective usage patterns may be partially captured here. Negation in particular is a known failure mode for bag-of-words models: the word "not" changes the sentiment of whatever follows it, but a unigram model treats "not" and "good" as independent features. POS tags do not fully solve this, but they add structural information that may help at the margins.

Text statistics (textstats). A TextstatsTransformer extracts document-level statistics: tweet length, punctuation counts, and similar surface features. These are scaled with StandardScaler. I was curious about this one. Strongly negative tweets often use more punctuation and exclamation marks. Very short tweets are often reactive expressions. Very long tweets may be more analytical and thus closer to neutral. I expected this to perform poorly as a standalone feature but potentially to add value in combination with other features. The results for textstats are the most diagnostic of any single configuration in terms of revealing what surface-level signals carry and what they miss.

VADER lexicon (lexicon). A LexiconCountVectorizer counts matches against the VADER sentiment lexicon, scaled with StandardScaler. VADER was designed specifically for social media sentiment (Hutto and Gilbert, 2014), and it encodes human judgements about which words are positive, negative, and neutral, including common social media expressions, emoticons, and slang. This is probably the most directly relevant feature set for this task. I expected it to perform well. The key question is whether the lexicon covers enough of the actual vocabulary in the tweet_eval dataset, which was collected later than the original VADER development set.

Embeddings. A Model2VecEmbedder generates dense vector representations of each tweet using a pre-trained embedding model. Embeddings capture semantic similarity in a way that sparse count features cannot. Two tweets that use different words to ex-

press the same sentiment should produce similar embedding vectors. Whether this generalises well in a short-text, noisy-language domain is an open question. My understanding is that pre-trained embeddings reflect the distributional statistics of the corpus they were trained on. If that corpus is general-purpose text rather than social media, the embeddings may not reliably represent informal abbreviated Twitter language. This is the feature type whose results I found most interesting to interpret, and I think it is the most theoretically rich.

3.2 Task variants

The 3-class task (negative, neutral, positive) is the primary task. The binary task (negative vs positive only, with neutral tweets excluded) is run as a diagnostic. My instinct was to start with the simpler binary task and then ask what happens when neutral is added back. The motivation is simple: if classifier performance improves substantially on the binary task, it suggests that the difficulty of the 3-class problem is largely concentrated at the boundary between neutral and the other two classes. If performance does not improve much, the problem lies elsewhere. This is directly relevant to sub-question 4.

The binary task uses a separate, independently undersampled training and test split. The binary training set contains 14,186 tweets (7,093 per class), and the binary test set contains 1,131 tweets.

4 Results

The results are organised into four runs: RUN 1 (3-class × logistic regression), RUN 2 (3-class × decision tree), RUN 3 (binary × logistic regression), and RUN 4 (binary × decision tree). For each run, classification reports and confusion matrices are shown for all six feature configurations. A summary table and a 3-class vs binary comparison table are provided at the end of this section.

4.1 RUN 1: 3-class classification, logistic regression

The logistic regression results across six feature configurations are shown in the classification reports and confusion matrices for RUN 1. The unigram baseline achieves a macro F1 of 0.447. This is the reference point for the other configurations. I was initially surprised that simple word counts could get this far, but on reflection it makes sense: words like “love”, “hate”, “great”, and “terrible” appear in the top 200 tokens and carry obvious sentiment signals. The classifier is doing something real. The macro F1 would be 0.333 for a random three-class guesser. We are well above that.

What the unigram model cannot do is handle negation, context, or ambiguous vocabulary. Words that appear frequently in positive and negative contexts alike — “day”, “time”, “people” — will receive weights close to zero and contribute little. The model will make its predictions based on the small set of vocabulary items that are reliably associated with one class. This is fine as a baseline, but it leaves a lot on the table.

The VADER lexicon configuration is the most directly designed for this task. The lexicon encodes human judgements about sentiment-bearing words and includes social media specific expressions not found in general vocabulary lists. I expected it to outperform the unigram baseline, and the results should be examined closely when assessing which feature type captures the most useful signal. Whether it does outperform unigrams across the board, or only for certain classes, is itself informative: a gain on negative precision, for example, would suggest the lexicon is particularly helpful for identifying strong negative language.

Embeddings represent the most semantically rich representation. They should, in theory, capture meaning beyond surface vocabulary. Whether they outperform simpler features in a noisy short-text domain is an empirical question, and the answer from this dataset is informative about the limits of general-purpose semantic representations.

4.2 RUN 2: 3-class classification, decision tree

The decision tree results are dramatically weaker than logistic regression across most feature configurations. With unigram features and max_depth=3, the tree achieves a macro F1 of 0.277 — barely above the 0.333 floor of random chance, and not by much. It does this by predicting neutral for the majority of inputs — recall for neutral approaches 1.0, while recall for negative and positive is near zero. This is consistent with a shallow tree learning that "predict the most common class" is the least-bad rule when only three splits are available.

The decision tree is not simply a weaker classifier: it is a qualitatively different kind of failure. Logistic regression distributes its errors more evenly. The decision tree collapses to a near-trivial heuristic. The three-node depth constraint is the most likely cause, but it may also reflect that the feature representations being tested do not contain strongly discriminating single-variable split points. Sentiment is not a simple threshold on a single word count.

Accuracy for the decision tree (0.474) is nearly identical to logistic regression (0.470). This is the textbook demonstration of why accuracy is the wrong metric for imbalanced multi-class problems. A model that overwhelmingly predicts the most common class will achieve near-majority-class accuracy while being useless for the minority classes. Macro F1 catches this; accuracy does not. It is a useful lesson. I found running both classifiers side by side on this dataset more instructive than any textbook example could be.

4.3 RUN 3 and RUN 4: binary task

The binary task removes neutral tweets and asks the classifier to distinguish negative from positive only. If the 3-class difficulty is concentrated at the neutral boundary, performance should improve substantially here. I expected a significant gain.

The comparison between 3-class and binary macro F1 for each feature configuration is shown in the gain table at the end of Section 4. A large positive gain for most configurations would confirm that neutral is the primary source of difficulty. A small or

inconsistent gain would suggest that the negative–positive boundary is also problematic, possibly because of label inconsistency at both boundaries.

The decision tree performs differently in the binary setting. Without the neutral class as a majority-class anchor, the tree must find a split that distinguishes negative from positive. Whether this produces a more useful model — one where recall is more balanced between the two classes — or whether it simply shifts which class gets over-predicted depends on the feature distributions. This is one of the results I was most interested to see.

5 Discussion

5.1 Overall performance

The logistic regression baseline achieved a macro F1 of 0.447 on the 3-class task. This is below 0.5. It is also, frankly, about where I expected it to land given the feature set and the known difficulty of the task. Below 0.5 is not a good score, but it is not a meaningless one either. The model is doing something. It is not random. That matters. For a dataset with known annotation noise and a notoriously hard neutral class, a macro F1 in the 0.4–0.5 range is a plausible baseline.

A random classifier on a balanced three-class problem would achieve macro F1 of 0.333. A model that always predicts the majority class on a class-imbalanced problem would have high recall for one class and near-zero for the others, yielding a macro F1 well below 0.333. The logistic regression result of 0.447 is comfortably above both of these floors. The question is not whether the classifier works. It is why it does not work better, and what that tells us about the data.

The decision tree at 0.277 is harder to defend. The accuracy figure (0.474) is almost identical to logistic regression (0.470), which is the clearest possible illustration of why accuracy is the wrong metric here. The test set is 43.5% neutral. A model that says "neutral" for every input achieves 43.5% accuracy without learning anything. The decision tree is not quite doing this, but it is close. Macro F1 exposes this; accuracy conceals it.

5.2 Feature comparison

The six feature configurations produce different results, and the pattern across them is revealing. Simple unigram counts establish a baseline — the floor against which everything else is measured. Bigrams add local context but face sparsity challenges in short informal text. The uni+pos combination adds grammatical structure. Text statistics capture surface properties that may correlate with emotional intensity. VADER lexicon features are purpose-built for this task. Embeddings provide the richest semantic representation, though whether that richness translates into better predictions on short noisy text is not obvious in advance.

I expected the VADER lexicon to be the top performer. The VADER lexicon was designed specifically for social media text and encodes human judgements about sentiment. It includes slang, emoticons, and common expressions in ways that a general

vocabulary count cannot. Hutto and Gilbert (2014) demonstrated its effectiveness on Twitter data, and the present task is almost identical in character to the tasks they used for evaluation.

Embeddings capture semantic similarity, which means that synonyms and paraphrases of sentiment expressions should map to similar feature vectors. But embeddings trained on general corpora may not fully capture the informal, abbreviated, and context-dependent language of Twitter. A tweet like "can't even rn" expresses frustration that a general embedding model may not reliably encode as negative.

Text statistics deserve comment. Tweet length, punctuation count, and capitalisation patterns are crude proxies for emotional intensity, but they are real signals. Very short tweets are often reactive. High punctuation density often accompanies strong emotion. I would expect textstats to perform poorly alone but potentially to add value as a complementary feature when combined with lexical or lexicon features. Running textstats as an isolated feature configuration is a diagnostic choice: it tells us what surface properties alone can do, which helps set expectations for combined models.

One thing I found interesting when comparing feature configurations is the consistency of the per-class patterns. For logistic regression, the ordering negative < positive < neutral in per-class F1 tends to be stable across feature types even though the absolute F1 values vary. To me, this suggests the difficulty ordering of the classes is a property of the data, not of any particular feature representation. The negative class is harder regardless of what features you use. That is a data observation, not a modelling one.

5.3 What the binary task reveals

Sub-question 4 asks whether removing neutral substantially improves performance. I believe the answer from the binary task results addresses a core concern about label quality: is neutral the main source of difficulty, or is the problem more general?

If the gain from removing neutral is large and consistent across classifiers and feature types, this supports the hypothesis that the neutral label is underspecified — that it captures a diffuse middle ground that is genuinely hard to separate from mild negative and mild positive expression. This would be consistent with the SemEval annotation process, where neutral was the default label for tweets that did not clearly express a directional sentiment.

If the gain is small, or absent for some configurations, the implication is different. It would suggest that the negative–positive boundary is also noisy, or that the feature representations are not adequate to capture the distinction even in the absence of the neutral class.

The gain comparison table in Section 4 shows the 3-class and binary macro F1 for each feature and classifier combination, along with the difference. I find this table more informative than the raw F1 figures in isolation, because it speaks directly to the label quality question rather than just to classifier performance. A large consistent gain is evidence for the "neutral is the problem" hypothesis. A small or variable gain suggests the difficulty is more distributed.

There is an additional consideration. The binary task uses a different, smaller test set — the original validation set minus all neutral tweets. A gain in macro F1 from 3-class

to binary may partly reflect this change in evaluation set rather than purely a change in task difficulty. The gain should be interpreted with this in mind, though the direction of effect is unlikely to be reversed by the compositional difference.

5.4 Class-level analysis

The 3-class LR results show asymmetry across classes. Positive tweets achieved the highest F1, driven by high precision but only moderate recall: the model is conservative about predicting positive, but when it does it is often right. Negative tweets achieved the lowest F1, with very low precision and moderate recall: the model over-predicts negative, and most tweets it calls negative were actually neutral or positive. Neutral sat in the middle.

Neutral is not the worst-performing class. That surprised me. The conventional view — supported by the course notes and by Kenyon-Dean et al. (2018) — is that neutral is the hardest class because it lacks the distinctive lexical markers that characterise clearly positive or clearly negative sentiment. The results do not clearly support this. Neutral F1 exceeds negative F1. This may be because the neutral class is the largest in the test set, giving the model more evaluation signal. Or it may be that neutral tweets genuinely cluster around common, non-sentiment-loaded vocabulary that is well represented in the top-200 unigram features. Negative tweets may use more varied, context-dependent language.

The decision tree collapse is a separate phenomenon. It is not simply that the tree performs poorly on negative and positive. It performs near-zero on both. The tree has apparently learned that predicting neutral minimises loss on the training data even after undersampling. This is concerning to me. It suggests the feature representations do not present a clean split between neutral and the other classes at any single threshold value.

5.5 Label quality evidence

The central question of this project is not simply how good the classifier is, but what classifier performance tells us about the labels. The two are not the same thing. Not even close.

Low macro F1 is consistent with several explanations. The feature set may be inadequate. The classifier may be ill-suited to the task. The labels may be noisy. All three are probably true to some degree here. I genuinely cannot tell which dominates.

The negative class stands out. Low precision means the model is generating many false positive predictions for negative sentiment. If the model predicts negative based on reasonable lexical cues but the gold label is neutral, this suggests cases where annotators labelled a borderline tweet as neutral rather than negative. The SemEval annotation guidelines used neutral as the default for tweets that did not clearly express positive or negative sentiment. This may have produced a diffuse, underspecified neutral category that captures genuine neutrality alongside mild negative expression that annotators were uncertain about.

Kenyon-Dean et al. (2018) argue that sentiment classification in tweets is complicated precisely because sentiment is a property of the author's relationship to the content, not just of the words used. The same phrase — "not bad at all" — may be positive or ironic depending on context. Unigram count features, which discard word order, negation scope, and context, are poorly equipped to capture these distinctions. But the hard cases may also involve genuine ambiguity that even human annotators would disagree on, and the low classifier performance may be tracking that ambiguity rather than merely reflecting model inadequacy.

The low negative precision may be an alignment signal. The model and the annotators share a rough understanding that certain vocabulary is negative — the model predicts negative, and in many cases the tweet probably is negative in some sense — but the annotators labelled it neutral, possibly because the expression was not strong enough to trigger the positive or negative label. If we had access to per-annotator votes rather than only the majority-vote outcome, we could test whether these tweets had high disagreement rates. The absence of that information is a genuine limitation.

5.6 Misclassification analysis

Notebook section 4.2 produces a systematic breakdown of model predictions by true and predicted class. Examining the misclassified tweets gives a ground-level view of where the model fails, and why.

Some misclassifications are model failures. A tweet containing strongly negative language predicted as neutral is likely a case where the relevant tokens fell outside the top-200 vocabulary, or where negation flipped the meaning of otherwise neutral words. These are feature engineering problems.

Other misclassifications are more interesting. When I looked at tweets predicted as negative but labelled neutral, a recurring pattern emerged: tweets expressing mild dissatisfaction, slight frustration, or low-key complaints — the kind of expression that sits at the boundary between neutral and negative in everyday language. These are the cases where I would expect human annotators to disagree. The model is not obviously wrong. The label may not be clearly right. That ambiguity is the point.

Similarly, some tweets predicted as positive but labelled neutral appear to express genuine enthusiasm in informal or restrained language. The model picks up the positive vocabulary but the gold label reflects an annotator who read the tweet as reporting rather than expressing. These borderline cases are exactly what the central research question is about.

The most consistently misclassified tweet types across all feature configurations are those that involve sarcasm, irony, or highly context-dependent expressions. A tweet like "great job guys" directed at a team that just lost can be positive, neutral, or sarcastic. No unigram feature set can resolve this. No bigram feature set can either. These represent a hard ceiling on what surface-form features can achieve. A human reading the same tweet with knowledge of the context — which sport, which team, which result — would resolve the ambiguity immediately. The classifier has none of that. It is working only from the words. That is a real constraint.

This is not a criticism of the approach. It is the nature of the problem. The question is whether the gold-standard labels also required contextual knowledge that crowdsourced annotators did not always have. If annotators were shown the tweet in isolation, without the thread context or the event context, they may have defaulted to neutral for tweets they could not confidently label positive or negative. That is exactly the kind of label noise that would explain the patterns observed here.

5.7 Limitations and future directions

The analysis has several limitations.

The feature configurations tested here are all applied independently. No multi-feature combinations are evaluated. A combined model — for example, unigrams plus VADER lexicon plus text statistics — might outperform any single configuration, and the interaction effects between feature types are not captured here. Combining features is a natural next step and one that the pipeline architecture supports directly through its FeatureUnion design. The cost of this analysis is missing those interactions; the benefit is a clear additive comparison of what each feature type contributes.

The decision tree hyperparameter choice of max_depth=3 is interpretable but severely limiting. A deeper tree might produce substantially better results. It would, however, be harder to interpret and more prone to overfitting on a training set of this size. The shallow constraint is a deliberate trade-off between performance and interpretability that is appropriate for the diagnostic purpose of this analysis.

The embeddings used here are from a pre-trained general-purpose model. Fine-tuning on Twitter data, or using a model pre-trained on social media text, might produce substantially different results. The generalisability of embedding-based representations to informal short text is a known limitation of off-the-shelf models. A model like BERTweet, pre-trained on 850 million tweets, would likely produce more useful tweet embeddings than a general-corpus model.

The analysis is constrained to the validation split. The held-out test split (12,284 tweets) has not been used and would provide a more reliable estimate of generalisation performance. The validation set has only 312 negative examples, which limits the precision of per-class estimates for that class. The wide confidence intervals around negative F1 are a real limitation, and results for that class should be treated as indicative rather than definitive.

Finally, the crowdsourced annotation process introduces a source of noise that cannot be removed by any classifier, no matter how well-designed. The gold standard labels represent majority-vote outcomes, not ground truth. Where annotators disagreed, the label is uncertain, and classifier performance on those cases is not a reliable measure of model quality. Separating these cases from clearly-labelled examples — perhaps by analysing inter-annotator agreement scores if available — would allow a more precise estimate of the irreducible difficulty of the task. This is probably the single most valuable extension of the analysis. Without knowing which labels are contested, we cannot distinguish “model failure” from “genuinely ambiguous tweet”.

References

Barbieri, F., Camacho-Collados, J., Espinosa-Anke, L., and Noves, L. (2020). TweetEval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1644–1650. Association for Computational Linguistics.

Hutto, C. J. and Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM 2014)*. AAAI Press.

Kenyon-Dean, K., Ahmed, E., Bhosale, S., Brooks, B., Laframboise, C., Roper, F., Singh, S., Cheung, J. C. K., and Precup, D. (2018). Sentiment analysis: It's complicated! In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1886–1895. Association for Computational Linguistics.

Nash, C. (2015). Atolls in the ocean—canaries in the mine? Australian journalism contesting climate change impacts in the Pacific. *Pacific Journalism Review*, 21(1):79–97.

Rosenthal, S., Farra, N., and Nakov, P. (2017). SemEval-2017 task 4: Sentiment analysis in twitter. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 502–518, Vancouver, Canada. Association for Computational Linguistics.